

Atividade 4 - Compressão, Representação e Descrição

Aluno: Vitor Fontenele de Oliveira Linhares - 1700778

Repositório

Imports

```
In [20]: import numpy as np
import cv2
import matplotlib.pyplot as plt
from pathlib import Path
```

Setup

```
In [21]: plt.rcParams['image.cmap'] = 'gray'

ARTEMIS_PATH = Path('artemis.jpg')
CRATERS_PATH = Path('craters_moon.jpg')
EARTH_SUN_PATH = Path('earth_and_sun.jpg')
SUN_PATH = Path('sun.jpg')
OUT_DIR = Path('outputs')

def load_image(path):
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f'Imagem não encontrada: {path.resolve()}')
    bgr = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if bgr is None:
        raise ValueError(f'Não foi possível ler a imagem: {path.resolve()}')
    if bgr.ndim == 2:
        return bgr
```

```
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)

def to_gray(img):
    if img.ndim == 2: return img
    return (0.299 * img[:, :, 0] + 0.587 * img[:, :, 1] + 0.114 * img[:, :, 2]).astype(np.float32)

def to_bin(img, threshold=128):
    if img.ndim == 3:
        img = to_gray(img)
    return (img >= threshold).astype(np.uint8) * 255

def save_image(filename, img, out_dir=OUT_DIR):
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    out_path = out_dir / filename
    arr = np.clip(img, 0, 255).astype(np.uint8)
    if arr.ndim == 3:
        arr = cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
    cv2.imwrite(str(out_path), arr)
    return out_path

def show_grid(images, titles, cols, figsize=None):
    n = len(images)
    rows = (n + cols - 1) // cols

    if figsize is None:
        figsize = (cols * 4, rows * 4)

    _, axes = plt.subplots(rows, cols, figsize=figsize)
    axes = axes.flatten() if n > 1 else [axes]

    for i in range(len(axes)):
        if i < n:
            axes[i].imshow(images[i], vmin=images[i].min(), vmax=images[i].max())
            axes[i].set_title(titles[i])
            axes[i].axis('off')

    plt.tight_layout()
    plt.show()
```

```
def initialize():
    artemis = load_image(ARTEMIS_PATH)
    earth_and_sun = load_image(EARTH_SUN_PATH)
    sun = load_image(SUN_PATH)
    craters_moon = load_image(CRATERS_PATH)

    artemis_bin = to_bin(artemis)
    earth_and_sun_bin = to_bin(earth_and_sun)
    sun_bin = to_bin(sun)

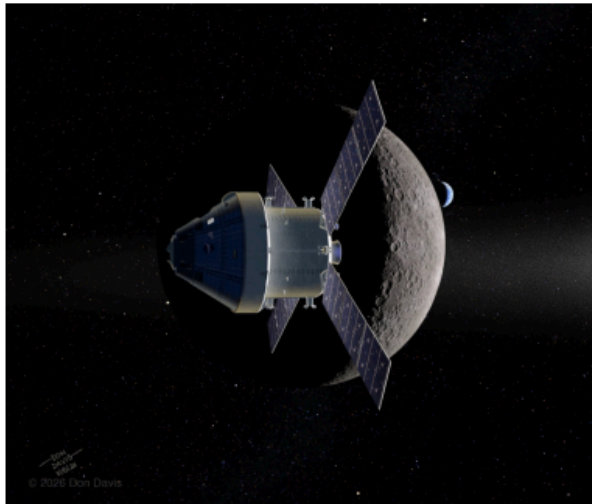
    ## Visualmente parece já estar em cinza (imagem do Watershed), mas é melhor prevenir do que remediar :)
    craters_moon_gray = to_gray(craters_moon)

    save_image('artemis_bin.jpg', artemis_bin, OUT_DIR)
    save_image('earth_and_sun_bin.jpg', earth_and_sun_bin, OUT_DIR)
    save_image('sun_bin.jpg', sun_bin, OUT_DIR)
    save_image('craters_moon_gray.jpg', craters_moon_gray, OUT_DIR)

    show_grid(
        [artemis, earth_and_sun, sun, artemis_bin, earth_and_sun_bin, sun_bin],
        ['Artemis', 'Earth and Sun', 'Sun', 'Artemis Bin', 'Earth and Sun Bin', 'Sun Bin'],
        cols=3
    )

initialize()
```

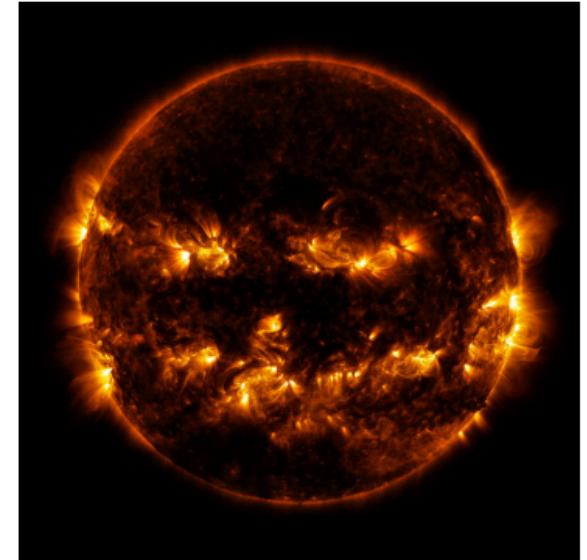
Artemis



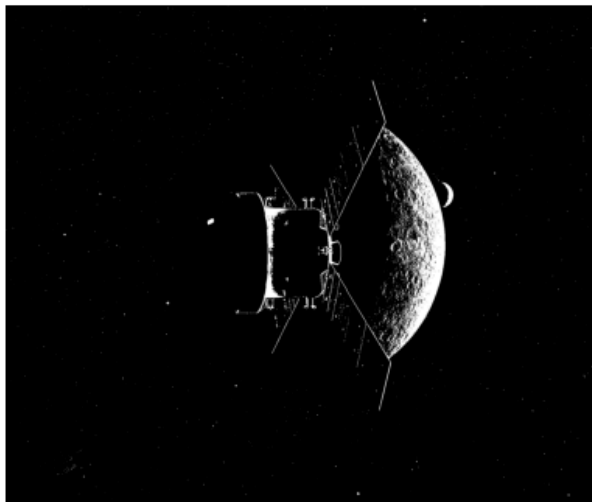
Earth and Sun



Sun



Artemis Bin



Earth and Sun Bin



Sun Bin



Questão 1

Operações Base

```
In [22]: def erosion(img, kernel):
    h, w = img.shape
    pad = kernel // 2
    out_img = np.zeros_like(img)

    padded_img = np.pad(img, pad, mode='edge')
    expected_sum = kernel * kernel * 255

    for y in range(h):
        for x in range(w):
            window = padded_img[y:y + kernel, x:x + kernel]

            if np.sum(window) == expected_sum:
                out_img[y, x] = 255

    return out_img

def dilation(img, kernel):
    h, w = img.shape
    pad = kernel // 2
    out_img = np.zeros_like(img)

    padded_img = np.pad(img, pad, mode='edge')

    for y in range(h):
        for x in range(w):
            window = padded_img[y:y + kernel, x:x + kernel]

            if np.sum(window) > 0:
                out_img[y, x] = 255

    return out_img

def opening(img_bin, kernel_size):
    eroded = erosion(img_bin, kernel_size)
    return dilation(eroded, kernel_size)
```

```
def closing(img_bin, kernel_size):  
    dilated = dilation(img_bin, kernel_size)  
    return erosion(dilated, kernel_size)
```

Retornar Tupla da Morfologia

```
In [23]: def process_morphology(img, kernel):  
    erosion_result = erosion(img, kernel)  
    dilation_result = dilation(img, kernel)  
    opening_result = opening(img, kernel)  
    closing_result = closing(img, kernel)  
  
    return erosion_result, dilation_result, opening_result, closing_result
```

Rodando

```
In [26]: images_bin = [  
    load_image(OUT_DIR / 'earth_and_sun_bin.jpg'),  
    load_image(OUT_DIR / 'sun_bin.jpg')  
]  
images_names = ["Earth_and_Sun", "Sun"]  
kernels = [3, 5, 15]  
  
show_grid(images_bin, images_names, cols=2)  
  
for idx in range(len(images_bin)):  
    img_bin = images_bin[idx]  
    name = images_names[idx]  
  
    for k_size in kernels:  
        print(f"{name} | Kernel: {k_size}x{k_size}...")  
  
        eroded, dilated, opened, closed = process_morphology(img_bin, k_size)  
  
        titles = [  
            f"Erosão {k_size}x{k_size}",  
            f"Dilatação {k_size}x{k_size}",  
            f"Abertura {k_size}x{k_size}",
```

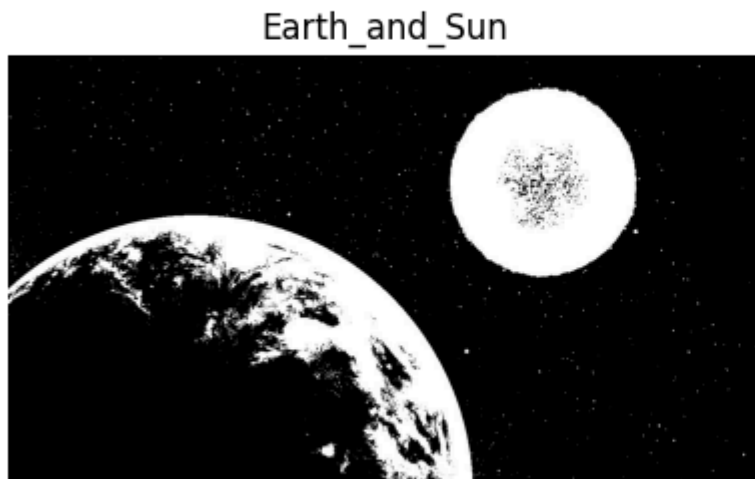
```

        f"Fechamento {k_size}x{k_size}"
    ]

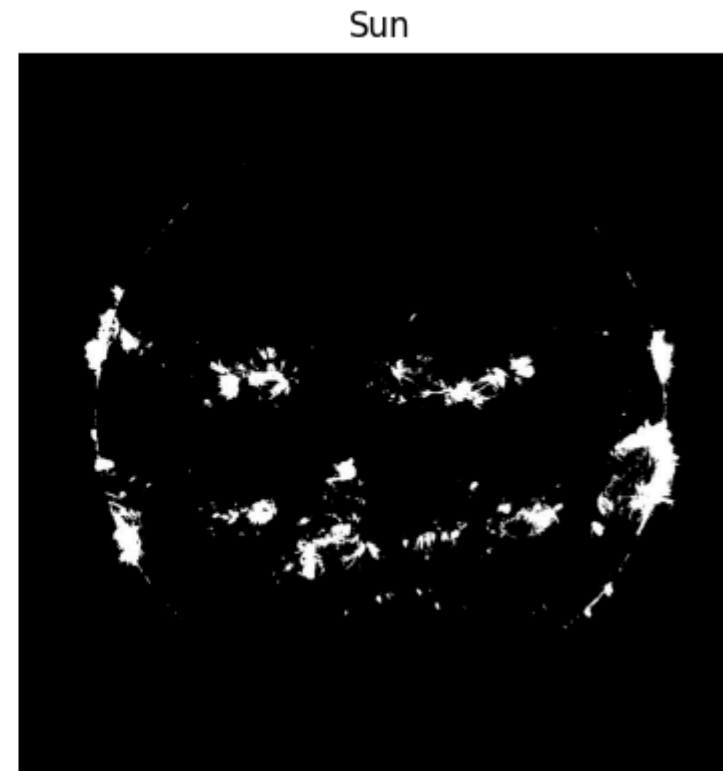
    save_image(f"{name.lower()}_erosion_{k_size}x{k_size}.jpg", eroded, OUT_DIR)
    save_image(f"{name.lower()}_dilation_{k_size}x{k_size}.jpg", dilated, OUT_DIR)
    save_image(f"{name.lower()}_opening_{k_size}x{k_size}.jpg", opened, OUT_DIR)
    save_image(f"{name.lower()}_closing_{k_size}x{k_size}.jpg", closed, OUT_DIR)

    show_grid([eroded, dilated, opened, closed], titles, cols=4)

```



Earth_and_Sun | Kernel: 3x3...





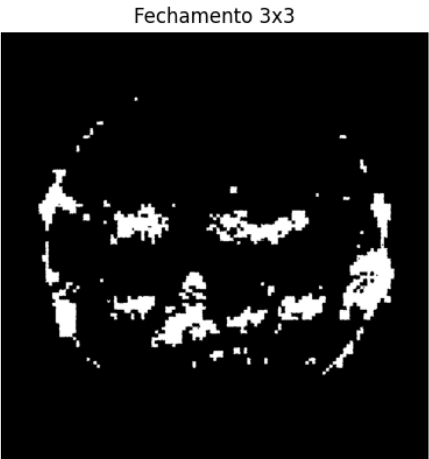
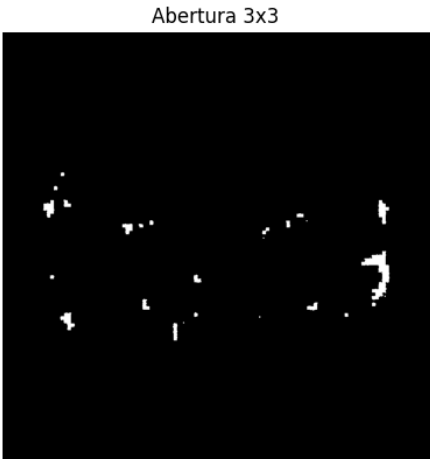
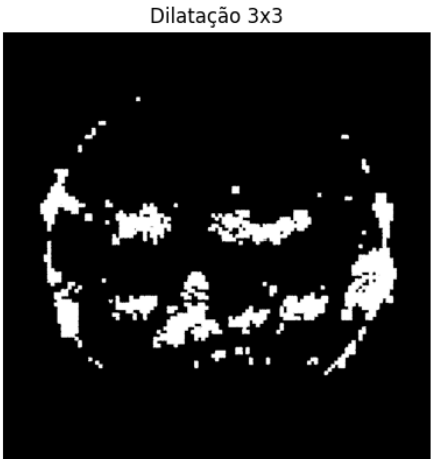
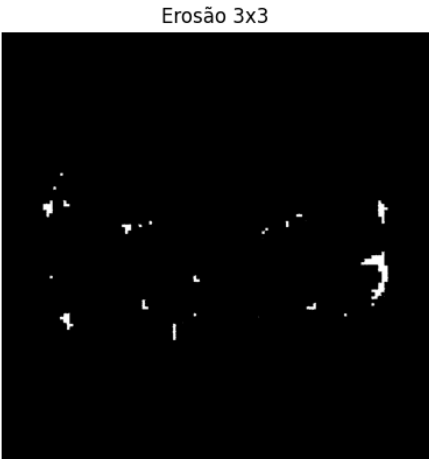
Earth_and_Sun | Kernel: 5x5...



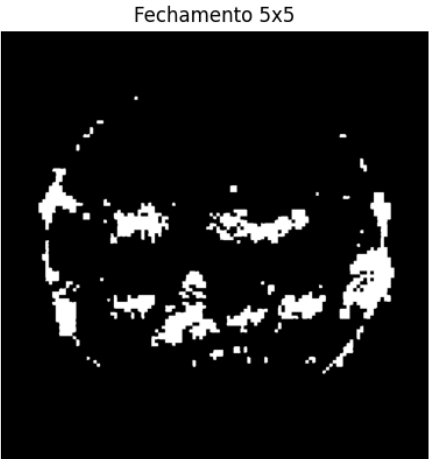
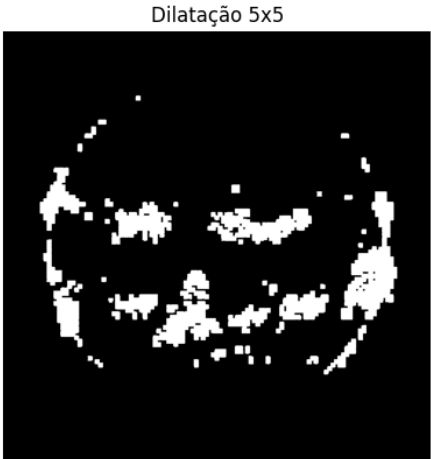
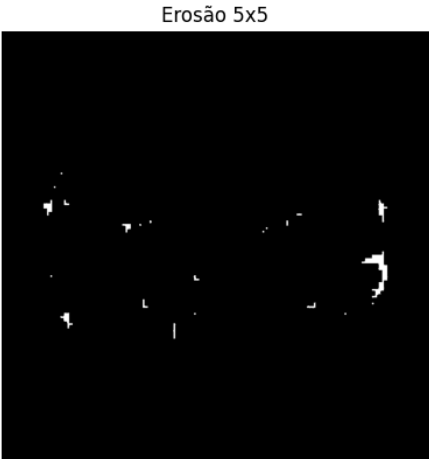
Earth_and_Sun | Kernel: 15x15...



Sun | Kernel: 3x3...



Sun | Kernel: 5x5...



Sun | Kernel: 15x15...

